OBSCURE — The Builder's Browser

ZORQUELZ TECHNOLOGIES

Stage 1 Tracer Bullet Engineering & Reverse-Engineering Guide

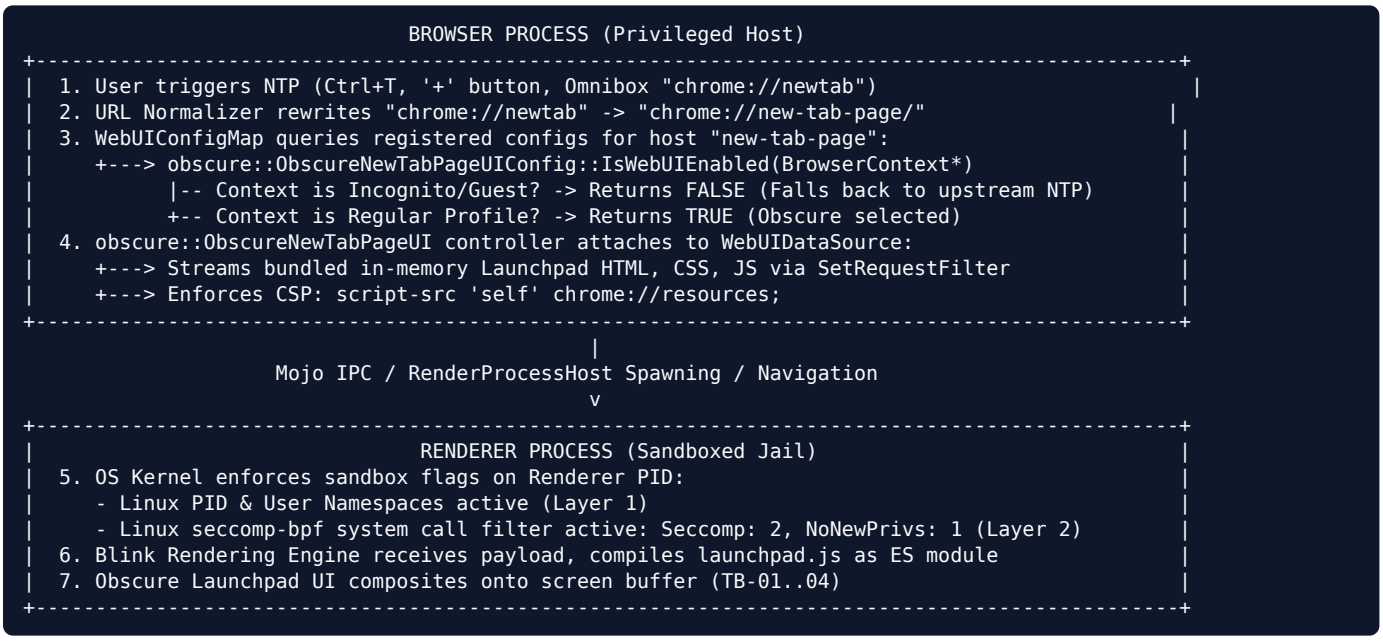
Target Audience:	Founder & CEO (Zorquelz)	**Control Branch:**	`feature/pr-1-qa-corrections`
Author / Lead:	Antigravity (Implementation Lead)	**Chromium Source Pin:**	Tag `155.0.8059.39` (`3ff7ac5a9...`)
Chief Architect:	ChatGPT	**Stage 1 Status:**	**IMPLEMENTATION IN PROGRESS**
Canonical Doc:	`docs/features/stage1-tracer-bullet/`	**Stage 2 Status:**	**STRICTLY BLOCKED (ADR 0002 PROPOSED)**

1. Executive Summary & Subsystem Architecture

This guide is the permanent reference for **Stage 1: New Tab Tracer Bullet**. It documents the complete architectural wiring, exact source code line-by-line, execution flow, security sandbox invariants, and defensible technical rationale.

> **Tracer Bullet Invariant:** Validates the complete vertical integration slice (Build System → WebUI Configuration → Sandboxed Renderer Delivery → Blink Paint) while strictly constraining external Chromium diffs to ≤10 lines and deferring all native persistence to Stage 2.

2. End-to-End System Topology & Control Flow



3. Exact Source Code & Reverse-Engineering Specification

A. Upstream Chromium Integration Hooks (≤10 External Lines)

- **Hook HK-01 (`chrome/browser/ui/webui/chrome_web_ui_configs.cc`):** Registers `ObscureNewTabPageUIConfig` inside `RegisterChromeWebUIConfigs()` substituting upstream `NewTabPageUIConfig` on `chrome::kChromeUINewTabPageHost`.
- **Hook HK-02 (`chrome/browser/BUILD.gn`):** Appends `"//chrome/browser/obscure"` to `source_set("browser")` public dependencies.

B. Isolated Obscure Module (`//chrome/browser/obscure/`)

Over 98% of Obscure's native code lives in this directory. If reverse engineering this subsystem, these are the canonical files:

- `BUILD.gn`: Declares `source_set("obscure")` depending on `//base`, `//content/public/browser`, and `//chrome/browser/profiles:profile`.

- `ui/webui/obscure_new_tab_page_ui_config.h/.cc`: Inherits from `content::WebUIConfig`. In `IsWebUIEnabled()`, checks `!profile->IsOffTheRecord()` guaranteeing zero Obscure UI execution in Incognito/Guest.
- `ui/webui/obscure_new_tab_page_ui.h/.cc`: Inherits from `content::WebUIController`. Creates `content::WebUIDataSource` on `chrome::kChromeUINewTabPageHost` and registers `SetRequestFilter` streaming compiled in-memory Launchpad HTML, CSS, and JS using `base::RefCountedString`.

4. Security Sandbox Topology & Boundary Protections

Layer	Enforcement Mechanism	Security Invariant Established
Layer 1 Sandbox	Linux PID, Mount, and Network Namespaces	Renderer process cannot view foreign PIDs, access host filesystems, or establish raw network sockets.
Layer 2 Sandbox	Seccomp-BPF Syscall Filter (`Seccomp: 2` , `NoNewPrivs: 1`)	Blocks unauthorized system calls (e.g. `execve`). Privilege escalation is strictly forbidden by the kernel.
Content Security Policy	`script-src 'self' chrome://resources;`	Strictly forbids inline scripts and `eval()` . All JavaScript must execute from isolated modules.
Incognito Isolation	`!profile->IsOffTheRecord()` Guard	Zero Obscure UI rendered in Incognito/Guest windows; automatically falls back to Chromium's native private splash screens.

5. Founder Mastery & Technical Defense Scenarios

> **Key Architectural Principle: "The Shim Pattern"**
> Obscure avoids the fatal trap of large Chromium forks (which drown in merge conflicts) by keeping its external diff footprint under 10 lines. Feature logic lives in an isolated module; upstream files only receive minimal registration calls.

Realistic Technical Inquiries & Defensible Answers

Q1: Why not build Obscure as an Electron app or a Chrome Extension?

Defensible Answer: Electron bundles a separate Node.js runtime and full Chromium instance per app, resulting in severe RAM inflation and an insecure sandbox model where renderer processes can directly touch the operating system. Extensions cannot modify browser network isolation, customize internal memory caching, or integrate system-level devtools without fighting Chrome Web Store permission constraints. Obscure is built directly into Chromium's native C++ engine, preserving full multi-process sandboxing while providing low-level engine control.

Q2: How does Obscure deliver assets in Stage 1 without touching Chromium's .pak resource files?

Defensible Answer: Chromium's `content::WebUIDataSource` provides a `SetRequestFilter` API that accepts a request path and returns an in-memory `base::RefCountedString` buffer. By serving compiled Launchpad HTML, CSS, and JS strings directly through this filter, we bypass editing upstream `.grd` resource files and `chrome_paks.gni`, keeping Stage 1 external diffs strictly under 10 lines.

Q3: What do the flags `Seccomp: 2` and `NoNewPrivs: 1` mean?

Defensible Answer: `NoNewPrivs: 1` is a Linux kernel bit ensuring that neither the process nor its children can ever acquire additional privileges (e.g. via setuid binaries). `Seccomp: 2` indicates filtering mode: the kernel validates every syscall against a Berkeley Packet Filter program, killing the process instantly if an unauthorized syscall is attempted.

6. Acceptance Verification Matrix (Test Plan 0003)

ID	Test Objective	Execution Command	Expected Result
TB-01..04	Launchpad Navigation & DOM	`./out/Stable155/chrome --headless=new --dump-dom chrome://new-tab-page/`	Launchpad cockpit HTML rendered
TB-05	Incognito Isolation	`./out/Stable155/chrome --headless=new --incognito --dump-dom chrome://new-tab-page/`	Standard Incognito splash; 0 Obscure UI
TB-06	Guest Isolation	`./out/Stable155/chrome --headless=new --guest --dump-dom chrome://new-tab-page/`	Standard Guest splash; 0 Obscure UI
TB-07	CSP Enforcement	`./out/Stable155/chrome --headless=new --enable-logging=stderr --v=1 chrome://new-tab-page/`	Zero CSP violations
TB-08	Sandbox Enforcement	`python3 scripts/check_sandbox.py` on renderer PID	`NoNewPrivs: 1`, `Seccomp: 2`
TB-10	Clean Patch Reversibility	`scripts/test_tb10_patch_revert.sh`	**PASS (VERIFIED)**